

CprE 3810, Computer Organization and Assembly Level Programming

Lab 1 Report

Student Name: _____ Isaiah Steele _____

Submit a typeset pdf version of this on Canvas by the due date. Refer to the highlighted language in the lab document for the context of the following questions.

[Part 3.1.c] **Think of three more cases and record them in your lab report.**

- Case 1:
 - inputs: $iW = 10$; $iLdW = 1$; $iX = 1$; $iY = 1$; (wait 2 cycles for outputs to be true)
 - outputs: $oX = 1$; $oY = 11$
- Case 2:
 - inputs: $iW = 11$; $iLdW = 1$; $iX = 2$; $iY = 5$; (wait 2 cycles for outputs to be true)
 - outputs: $oX = 2$; $oY = 27$ (I was born on the 27th; it is the day of my birth)
- Case 3:
 - inputs: $iW = 12$; $iLdW = 1$; $iX = 3$; $iY = 10$; (wait 2 cycles for outputs to be true)
 - outputs: $oX = 3$; $oY = 46$

[Part 3.1.e] **For labels 1, 7, 22, and 28, specify where (VHDL file and line number) these values are located – some will be found in more than one place. Also attempt to explain the functionality of each label as it occurs in the code**

1: In the attached diagram area, (1) is the whole of the MAC module, which is defined as having 4 inputs (iX , iW , $iLdW$, iY) aside from the clock on lines 29 – 32 of `TPU_MV_Element.vhd` and having 2 outputs (oY , oX) on lines 33 and 34. These inputs and outputs each have a width of 8 except for oY and $iLdW$. oY has a width of 32 bits, and $iLdW$ is a control line and is a single bit. The entire module functions as a single unit in a TPU MV element from a Google TPU.

7: In the attached diagram area, (7) is the label of the `g_Add1` module, which has two inputs (aside from the clock), iA and iB , and one output, oC . These are specified in file `Adder.vhd` on lines 33 and 34 for the inputs, and 35 for the output. The two inputs and the one output are 32 bits. This module adds the output of the multiplier module with the

22: In the attached diagram area, (22) is oQ , the output inside the `g_Weight` module, which is defined as a `DATA_WIDTH-bit std_logic_vector` (32 bits) on line 30 of `RegLd.vhd` and line 72 of `TPU_MV_Element.vhd`. It is used in line 96 of `TPU_MV_Element.vhd` and is mapped as s_W in the attached document as the output of `g_Weight`. oQ is also mapped as an input to the mux in `g_Weight`, which is specified on line 96 of `TPU_MV_Element.vhd`.

28: In the attached diagram area, (28) is the s_X1 output of `g_Delay` (specified in `g_Delay1` on line 106 of `TPU_MV_Element.vhd`) and into `g_Delay3` line called iD (specified on line 128 of `TPU_MV_Element.vhd` called). The line is defined as a `DATA_WIDTH-bit std_logic_vector` on line 29 of `Reg.vhd`.

[Part 3.1.f] Implement your test cases from part c into this testbench. Remember to include these in your waveform screenshot.

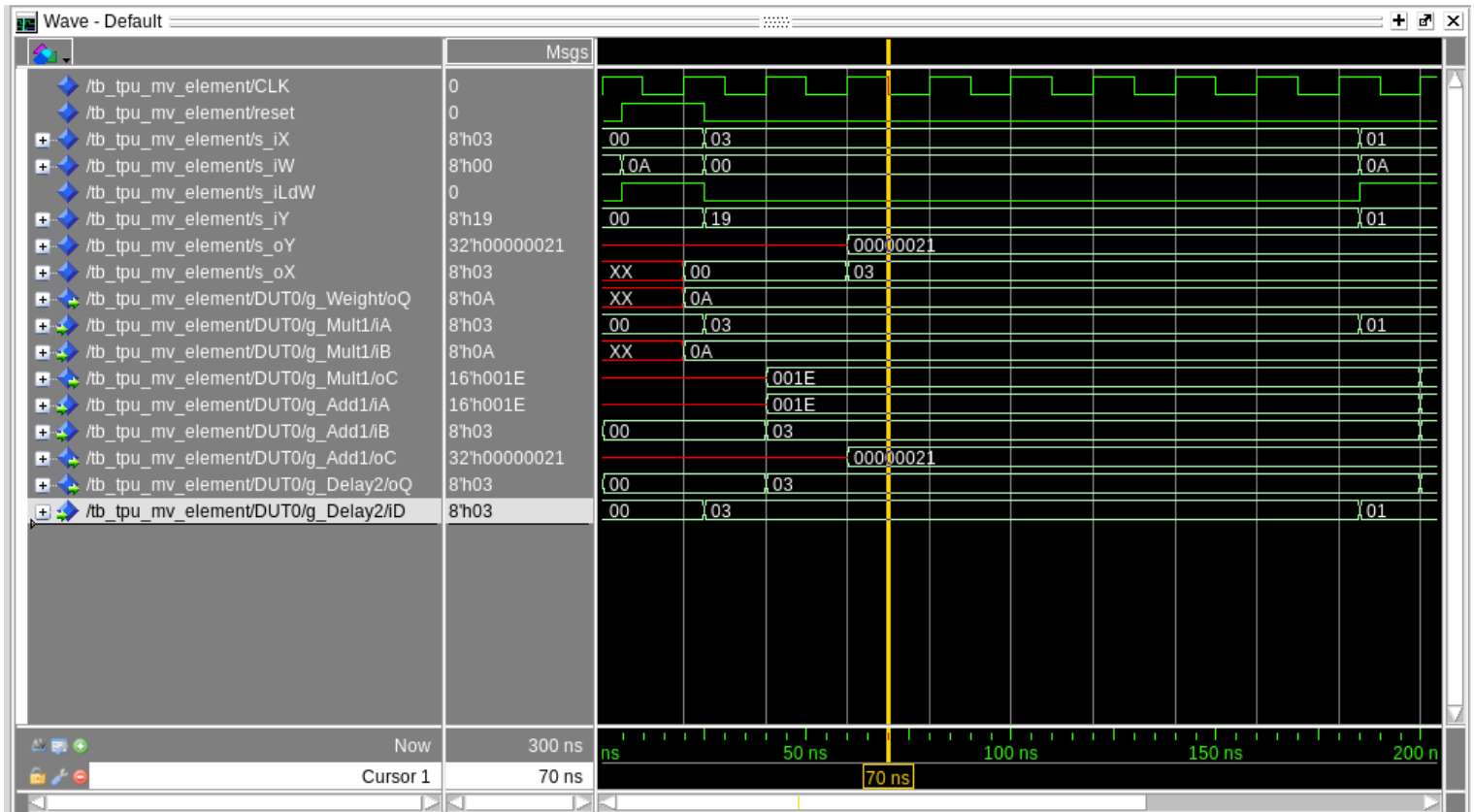
```
-- Test case 3:
-- Perform average example. iW=10, iLdW=1, iX=1, iY=1
s_iX    <= x"01";
s_iW    <= x"0A";
s_iLdW  <= '1';
s_iY    <= x"01";
wait for gCLK_HPER*2;
wait for gCLK_HPER*2;
-- Expect: o_Y output signal to be 11 = 1*10+1 and o_X output signal to be 1 after two positive edge of clock.

-- Test case 4:
-- Perform average example. iW=11, iLdW=1, iX=2, iY=5
s_iX    <= x"02";
s_iW    <= x"0B";
s_iLdW  <= '1';
s_iY    <= x"05";
wait for gCLK_HPER*2;
wait for gCLK_HPER*2;
-- Expect: o_Y output signal to be 27 = 2*11+5 and o_X output signal to be 2 after two positive edge of clock.

-- Test case 5:
-- Perform average example. iW=12, iLdW=1, iX=3, iY=10
s_iX    <= x"03";
s_iW    <= x"0C";
s_iLdW  <= '1';
s_iY    <= x"0A";
wait for gCLK_HPER*2;
wait for gCLK_HPER*2;
-- Expect: o_Y output signal to be 46 = 3*12+10 and o_X output signal to be 3 after two positive edge of clock.
```

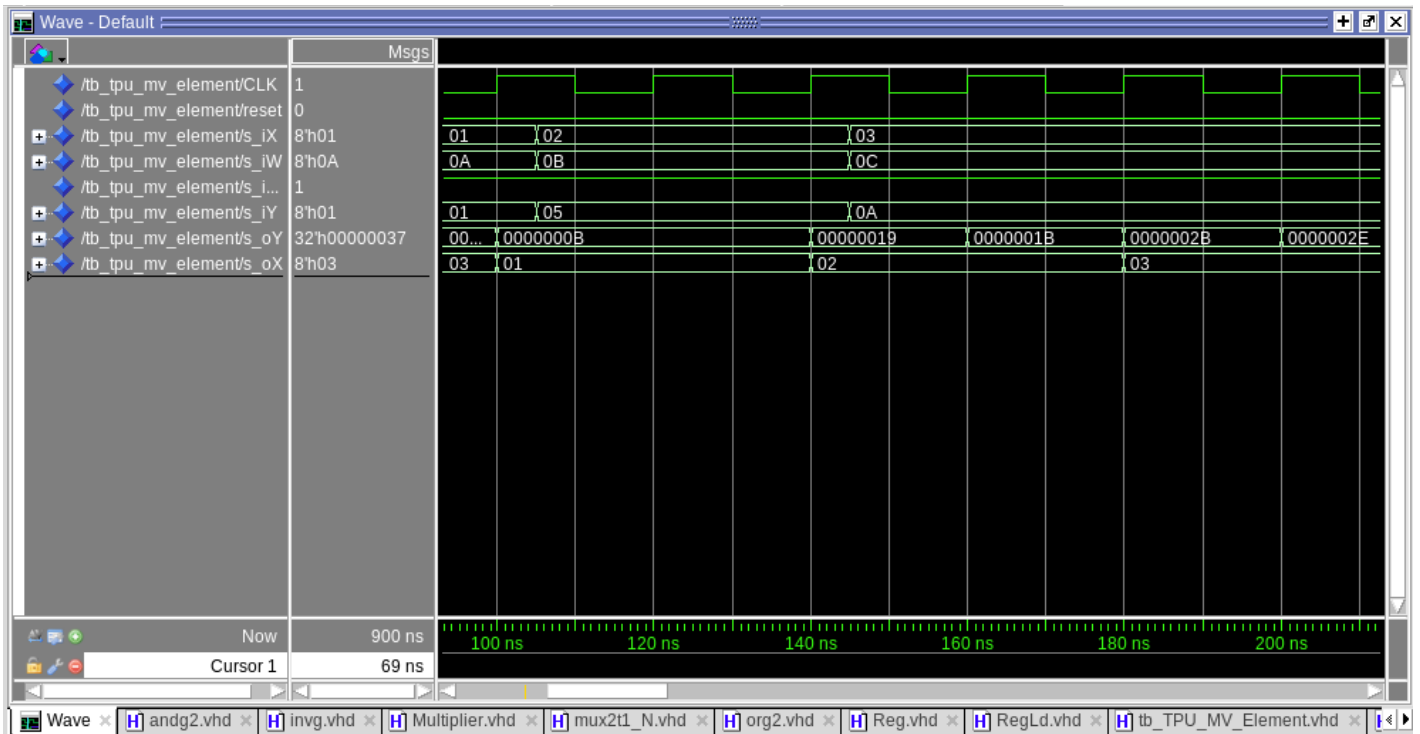
[Part 3.1.g.v] In your lab report, include a screenshot of the waveform. Describe, in plain English, any differences between what you expected and what the simulation showed.

(provided test case 2 was duplicated by me three times in order to aid debugging)



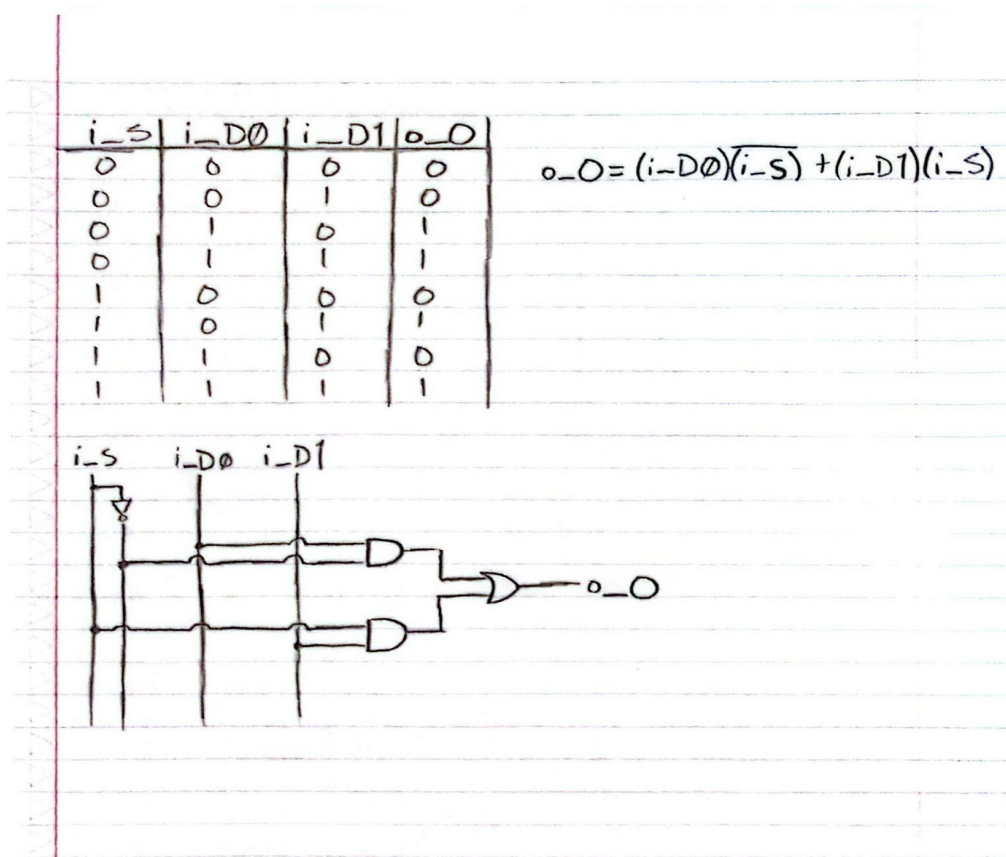
At 25 nanoseconds, the inputs of *s_iX*, *s_iW*, *s_iLdW*, and *s_iY* are set. However, the expected output on *oY* is not 55 as expected. It is 33. There is a similar problem with the rest of the test cases.

[Part 3.1.h] In your lab report, include a screenshot of the waveform. Describe, in plain English, how your waveform matches the expected result (e.g., reference the specific cycles and times). In your submission zip file, provide the completed *TPU_MV_Element.vhd* file in a folder called 'MAC'.

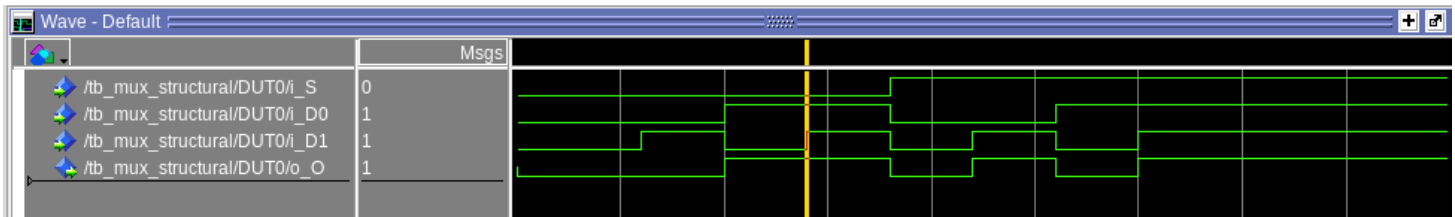


We can see that the *oY* line has the three correct outputs from my three tests (outputs of 0xB (for 11), 0x1B (for 27), and 0x2E (for 46)). These outputs come every two clock edges, which is correct according to my test cases (new inputs are given every 2 clock cycles). There are garbage results in-between the expected 2-clock-aligned outputs.

[Part 3.3.a] Draw the truth table, Boolean equation, and Boolean circuit equivalent (using only two-input gates) that implements a 2:1 mux. Include this in your lab report.

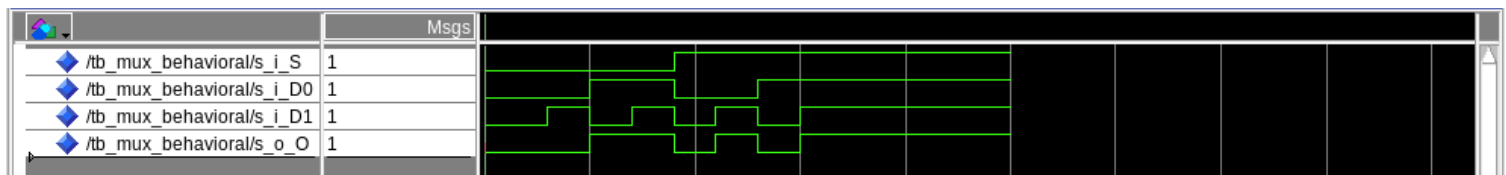


[Part 3.3.d] In your lab report, include a screenshot of the waveform. Make sure to label the screenshot with which module it is testing.



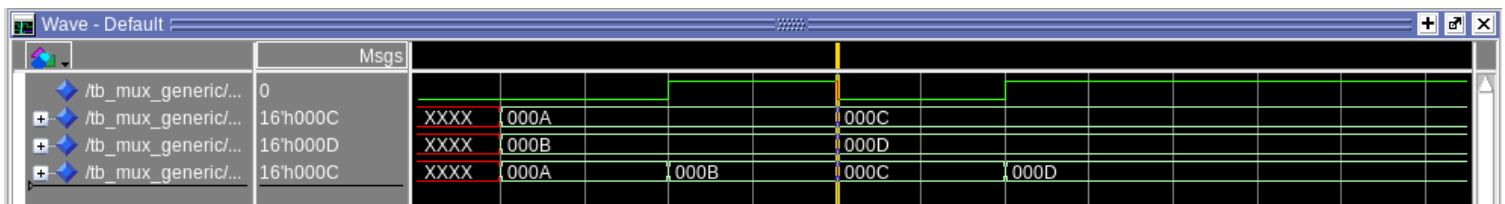
LABEL: Module “mux_structural” is being tested here.

[Part 3.3.e] Again, in your lab report, include a labeled screenshot of the waveform showing the dataflow mux implementation working.



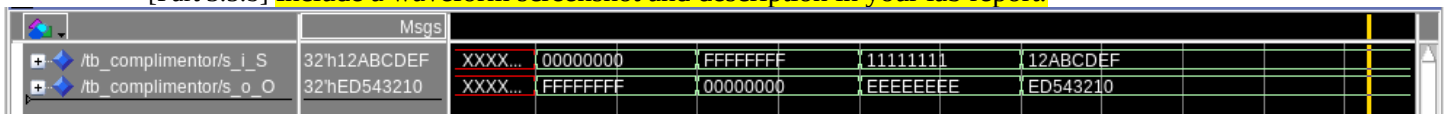
LABEL: Module “mux_behavioral” is being tested here.

[Part 3.4] Include a waveform screenshot and corresponding description demonstrating it is working correctly.



The test cases shown in the screenshot are four. Two test cases that have the hex numbers A and B as inputs, one with the select as 0 and the other as 1. The other two are similar, using C and D, and switch from 0 to 1. This produces the four outputs A, B, C, and D.

[Part 3.5.b] Include a waveform screenshot and description in your lab report.



Four test cases are seen here in the 32-bit configuration testing complimentor.hdl. The following hex values are tested: 0x00000000, 0xFFFFFFFF, 0x11111111, and 0x12ABCDEF. They are all inverted correctly, as is demonstrated on the waveform diagram.

[Part 3.6.a] A full adder takes three single-bit inputs and produces two single-bit outputs – a sum and carry for the addition of the three input bits. Draw the truth table, Boolean equation, and Boolean circuit equivalent (using only two-input gates) that implements a 1-bit full adder. Include this in your report.

i_A	i_B	i_C	o_S	o_C
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

o_S:

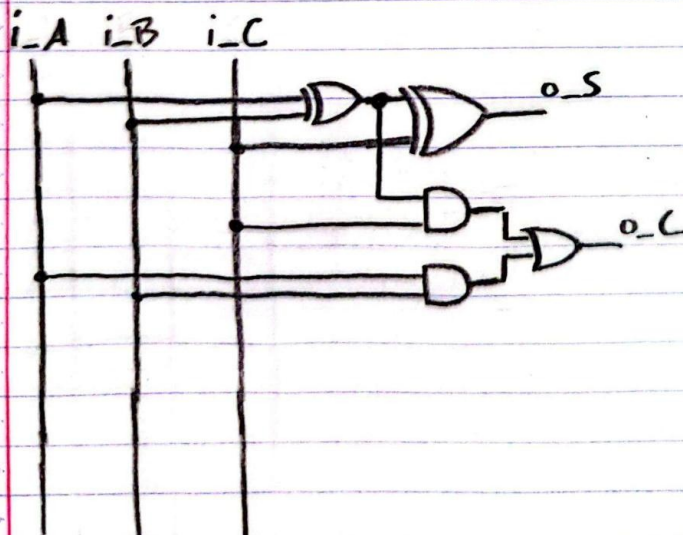
c \ AB	00	01	11	10
0	0	1	0	1
1	1	0	1	0

o_C

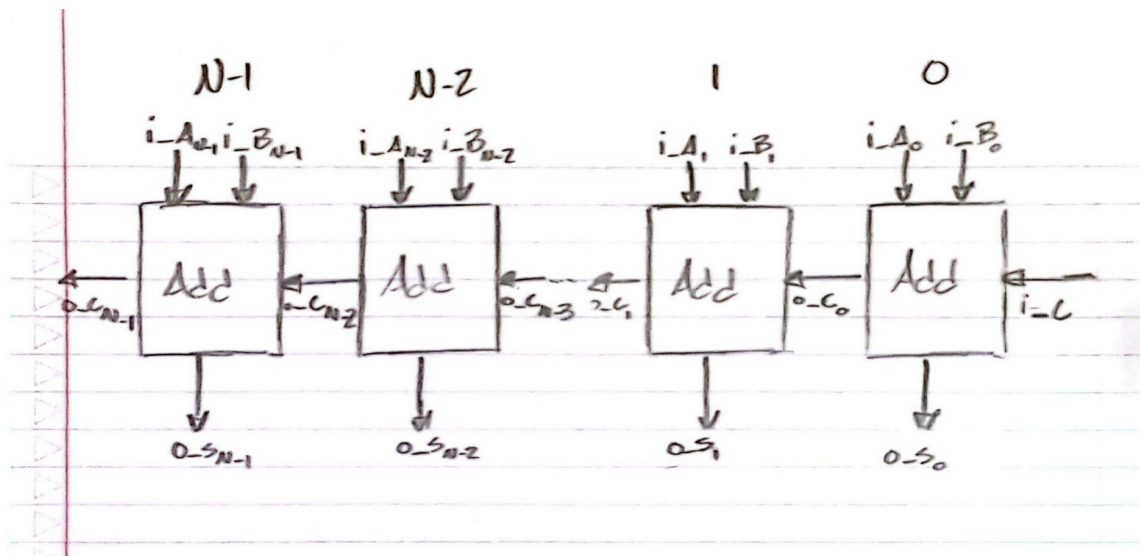
c \ AB	00	01	11	10
0	0	0	1	0
1	0	1	1	1

$$o_S = (\bar{A}\bar{B}\bar{C}) + (\bar{A}\bar{B}C) + (\bar{A}B\bar{C}) + (AB\bar{C}) = A \oplus B \oplus C$$

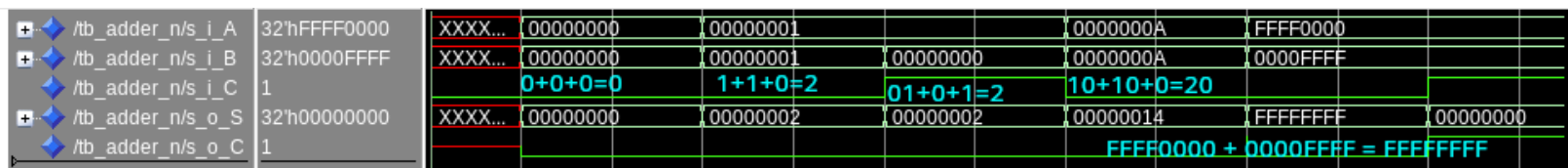
$$o_C = (AB) + (BC) + (AC)$$



[Part 3.6.c] Then draw a schematic of the intended design, including inputs and outputs and at least the 0, 1, N-2, and N-1 stages. Include this in your report.



[Part 3.6.d] Include an annotated waveform screenshot in your write-up.



[Part 3.7.a] Draw a schematic (don't use a schematic capture tool) showing how an N-bit adder/subtractor with control can be implemented using only the three main components designed in earlier parts of this lab (i.e., the N-bit inverter, N-bit 2:1 mux, and N-bit adder). How is the 'nAdd_Sub' bit used? Include this in your report.

The nAdd_Sub bit is used both as a selector for the complimented/not complimented mux (to decide whether subtraction is happening or not) and is also acting as a carry bit (incidentally, a carry bit when subtraction is used to facilitate 2s complement math).

